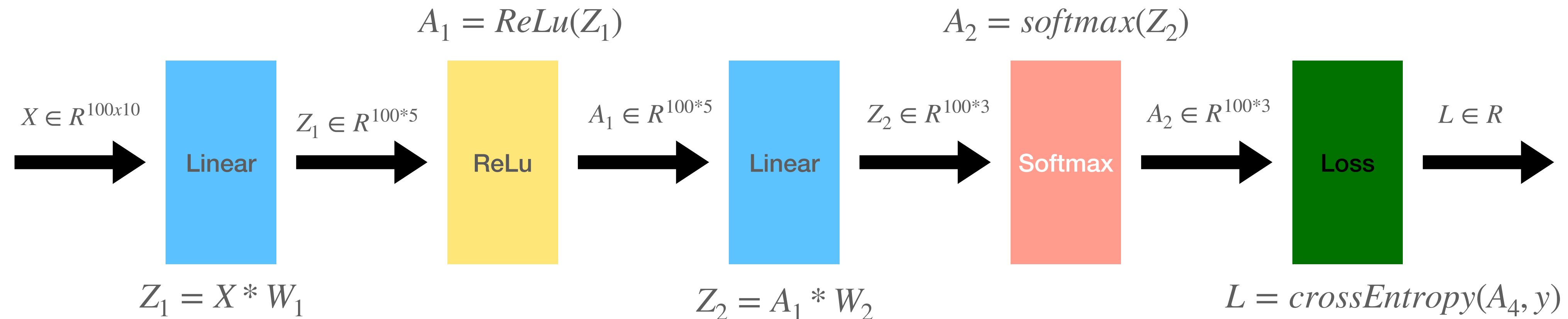


Autograd

Two layers Network



Input

$$X \in R^{100 \times 10}$$

$$y \in R^{100}$$

Parameters

$$W_1 \in R^{10 \times 5}$$

$$W_2 \in R^{5 \times 3}$$

Forward

$$Z_1 = X * W_1$$

$$A_1 = ReLu(Z_1)$$

$$Z_2 = A_1 * W_2$$

$$A_2 = softmax(Z_2)$$

$$L = crossEntropy(A_2, y)$$

Backward

$$dZ_2 = Z_2 - one_hot(y)$$

$$dW_2 = \frac{A_1^T * dZ_2}{m}$$

$$dA_1 = dZ_2 * W_2^T$$

$$dZ_1 = dA_1 * Relu_derivative(A_1)$$

$$dW_1 = \frac{X^T * dZ_1}{m}$$

Have to have full copy of weights to backward.

Update

$$W_2 = W_2 - \eta dW_2$$

$$W_1 = W_1 - \eta dW_1$$

Two layers Network

Input

$$X \in R^{100 \times 10}$$

$$y \in R^{100}$$

Parameters

$$W_1 \in R^{10 \times 5}$$

$$W_2 \in R^{5 \times 3}$$

Forward

$$Z_1 = X * W_1$$

$$A_1 = \text{ReLU}(Z_1)$$

$$Z_2 = A_1 * W_2$$

$$A_2 = \text{softmax}(Z_2)$$

$$L = \text{crossEntropy}(A_2, y)$$

Backward

$$dZ_2 = Z_2 - \text{one_hot}(y)$$

$$dW_2 = \frac{A_1^T * dZ_2}{m}$$

$$dA_1 = dZ_2 * W_2^T$$

$$dZ_1 = dA_1 * \text{Relu_derivative}(A_1)$$

$$dW_1 = \frac{X^T * dZ_1}{m}$$

Update

$$W_2 = W_2 - \eta dW_2$$

$$W_1 = W_1 - \eta dW_1$$

Have to have full copy of weights to backward.

PyTorch

Loss = model(input)

loss.backward()

optimizer.step()

Two layers Network

Input

$$X \in R^{100 \times 10}$$

$$y \in R^{100}$$

Parameters

$$W_1 \in R^{10 \times 5}$$

$$W_2 \in R^{5 \times 3}$$

Forward

$$Z_1 = X * W_1$$

$$A_1 = ReLu(Z_1)$$

$$Z_2 = A_1 * W_2$$

$$A_2 = softmax(Z_2)$$

$$L = crossEntropy(A_2, y)$$

Backward

$$dZ_2 = Z_2 - one_hot(y)$$

$$dW_2 = \frac{A_1^T * dZ_2}{m}$$

$$dA_1 = dZ_2 * W_2^T$$

$$dZ_1 = dA_1 * Relu_derivative(A_1)$$

$$dW_1 = \frac{X^T * dZ_1}{m}$$

Have to have full copy of weights to backward.

Update

$$W_2 = W_2 - \eta dW_2$$

$$W_1 = W_1 - \eta dW_1$$

```
# Generate dummy data: 100 samples, 10 features, 3 classes
X = np.random.randn(100, 10)
y = np.random.randint(0, 3, size=100)
```

```
# Initialize parameters
def init_params(input_size, hidden_size, output_size):
    W1 = np.random.randn(input_size, hidden_size) * 0.01
    b1 = np.zeros((1, hidden_size))
    W2 = np.random.randn(hidden_size, output_size) * 0.01
    b2 = np.zeros((1, output_size))
    return W1, b1, W2, b2
```

```
# Forward pass
def forward(X, W1, b1, W2, b2):
    Z1 = X.dot(W1) + b1
    A1 = relu(Z1)
    Z2 = A1.dot(W2) + b2
    A2 = softmax(Z2)
    return Z1, A1, Z2, A2
```

```
# Backward pass
def backward(X, y, Z1, A1, A2, W2):
    m = X.shape[0]
    y_one_hot = one_hot(y, A2.shape[1])

    dZ2 = A2 - y_one_hot
    dW2 = A1.T.dot(dZ2) / m
    db2 = np.sum(dZ2, axis=0, keepdims=True) / m

    dA1 = dZ2.dot(W2.T)
    dZ1 = dA1 * relu_derivative(Z1)
    dW1 = X.T.dot(dZ1) / m
    db1 = np.sum(dZ1, axis=0, keepdims=True) / m

    return dW1, db1, dW2, db2
```

```
# Training function
def train(X, y, input_size, hidden_size, output_size, epochs=1000, lr=0.01):
    W1, b1, W2, b2 = init_params(input_size, hidden_size, output_size)

    for epoch in range(epochs):
        # Forward
        Z1, A1, Z2, A2 = forward(X, W1, b1, W2, b2)

        # Loss
        loss = cross_entropy_loss(y, A2)

        # Backward
        dW1, db1, dW2, db2 = backward(X, y, Z1, A1, A2, W2)

        # Update
        W1 -= lr * dW1
        b1 -= lr * db1
        W2 -= lr * dW2
        b2 -= lr * db2

        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Loss: {loss:.4f}")

    return W1, b1, W2, b2
```

**How does backward do in
PyTorch?**

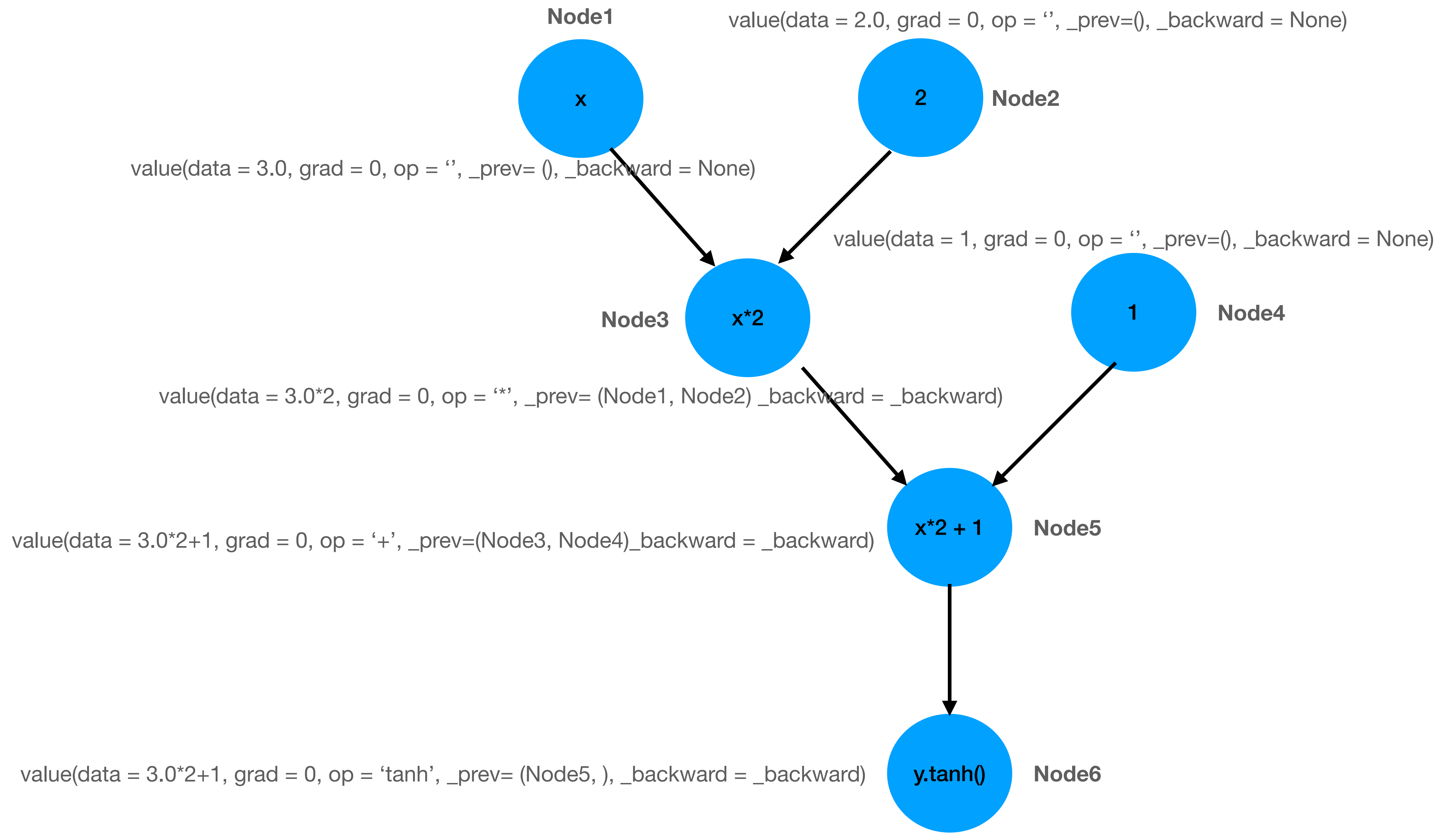
Autograd

```
# Example:  $f(x) = (x * 2 + 1).tanh()$   
x = Value(3.0)  
y = x * 2 + 1  
z = y.tanh()  
z.backward()
```

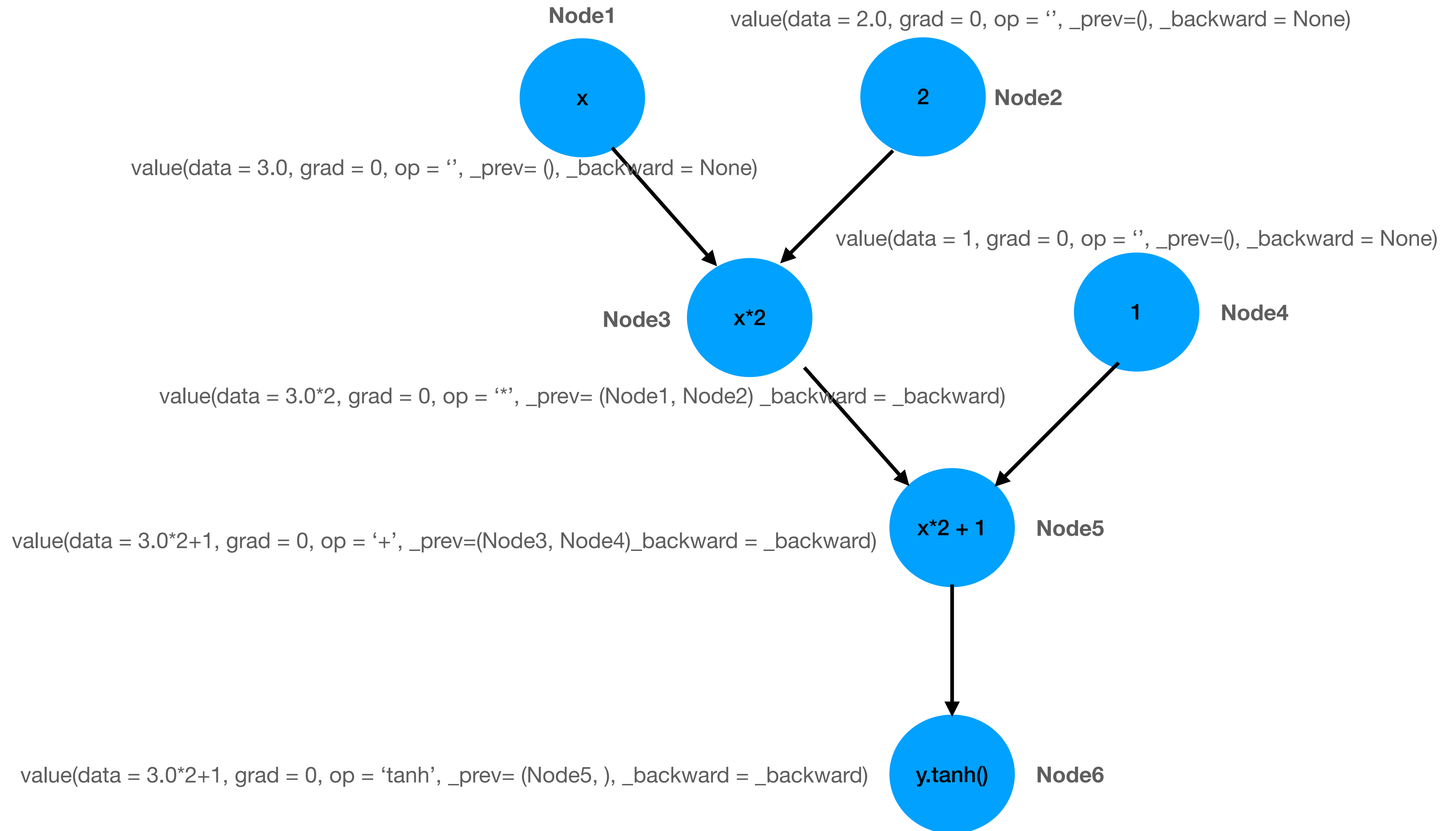
How to autograd:

- 1. Build computation graphs.**
- 2. Compute gradient via reverse-mode autodiff.**

Autograd



Autograd



When call backward by `z.backward()`, it will follow the nodes in the graph to compute gradient of each node.

Autograd

```
class value:
    def __init__(self, data, _children=(), _op=''):
        self.data = float(data)      # value of the node
        self.grad = 0.0              # gradient initialization to 0
        self._prev = set(_children)  # node that this node depends on
        self._op = _op               # operation that produced this node
        self._backward = lambda: None # function to compute local gradient

    def __add__(self, other):
        other = other if isinstance(other, value) else value(other)
        out = value(self.data + other.data, (self, other), '+')

        def _backward():
            self.grad += out.grad
            other.grad += out.grad

        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, value) else value(other)
        out = value(self.data * other.data, (self, other), '*')

        def _backward():
            self.grad += other.data*out.grad
            other.grad += self.data*out.grad

        out._backward = _backward
        return out

    def tanh(self):
        x = self.data
        t = np.tanh(x)
        out = value(t, (self,), 'tanh')

        def _backward():
            self.grad += (1 - t**2) * out.grad

        out._backward = _backward
        return out
```

```
# Example: f(x) = (x * 2 + 1).tanh()
x = Value(3.0)
y = x * 2 + 1
z = y.tanh()
z.backward()
```

```
def backward(self):
    # topological sort
    topo = []
    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)

    build_topo(self)
    self.grad = 1.0
    for node in reversed(topo):
        node._backward()
```